Milestone 2 Report

Predicting Gaming Addiction: Multi-Layer Perceptron and K-Means Clustering

392443 – 400305 – 399383

May 29, 2026

Keywords: *MLP, Backpropagation, K-Means, Model Selection, Bias–Variance*

1. INTRODUCTION

We continue the study begun in Milestone 1 on the same dataset of 2000 samples described by 13 numerical features of gaming habits (1600 train / 400 test), tackling the same two tasks: *(i)* regression of the addiction score (integer in $[0, 10]$) and *(ii)* classification of the addiction level into three classes {Low, Medium, High}. Here we implement two non-linear methods from scratch: a multi-layer perceptron and K-Means clustering. As before, we hold out 20% of the training set as a validation set (random split, fixed seed), fit on the remaining 80%, and select every hyperparameter on the validation metrics; the test set is used only once, for the final numbers reported in Table 1.

2. METHODS

2.1. Data preprocessing

Identical to Milestone 1: every feature is standardized with a z-score, $x_{\text{norm}} = (x - \mu)/\sigma$, where μ and σ are estimated on the training set only (to avoid leakage) and applied to both splits. No bias column is appended: the MLP carries explicit per-layer biases and K-Means operates directly on Euclidean distances.

2.2. Multi-layer perceptron

We implement a fully-connected network with L layers. Writing $a^{(0)} = x$ for the input, each layer computes a pre-activation and an activation:

$$s^{(l)} = a^{(l-1)} \left(W^{(l)}\right)^{\top} + b^{(l)}, \qquad a^{(l)} = \phi_l\left(s^{(l)}\right). \tag{1}$$

All hidden layers use ReLU, $\phi(s) = \max(s, 0)$, while the output depends on the task. For regression the output layer is linear (identity activation) and the network minimizes the mean-squared error (MSE). For classification the C output units pass through a sigmoid and are trained to match the one-hot target under MSE, the predicted class being the arg max of the outputs; a softmax + cross-entropy variant is also implemented. Weights are initialized à la Xavier, $W \sim \mathcal{N}(0,\ 1/\text{fan_in})$, and biases at zero. Training uses mini-batch stochastic gradient descent (batch size 64, 100 epochs, data reshuffled each epoch). Gradients follow from backpropagation: at the output layer $\delta^{(L)} = \nabla_{a^{(L)}} \mathcal{L} \odot \phi_L'(s^{(L)})$, propagated as $\delta^{(l)} = (\delta^{(l+1)} W^{(l+1)}) \odot \phi_l'(s^{(l)})$, with weight and bias gradients $\sum_n \delta^{(l)\top} a^{(l-1)}$ and $\sum_n \delta^{(l)}$. The learning rate η and the hidden architecture are the tuned hyperparameters.

2.3. K-Means clustering

K-Means partitions the training points into K clusters by minimizing the within-cluster sum of squares (inertia),

$$J = \sum_{k=1}^{K} \sum_{i:\, c_i = k} \left\| x_i - \mu_k \right\|^2, \tag{2}$$

through Lloyd's algorithm: assign each point to its nearest centroid, recompute each centroid as the mean of its assigned points, and iterate until the assignments stabilize or `max_iters` is reached. To turn this unsupervised method into a classifier, after convergence we label each cluster by the majority class of the training points it contains (arg max of the per-cluster label counts); a test point then receives the label of its nearest centroid's cluster. We compare two seedings: plain *random* initialization and *k-means++*, which spreads the initial centroids by drawing each new seed with probability proportional to its squared distance to the closest existing centroid.

3. EXPERIMENTS AND RESULTS

We select every hyperparameter on the validation split and read over/under-fitting directly from the gap between the train and validation curves, exactly as in Milestone 1.

3.1. MLP: learning rate (classification)

With a fixed $[64, 32]$ architecture we sweep η over $\{10^{-3}, \ldots, 5 \cdot 10^{-1}\}$ (Fig. 1). Validation accuracy peaks at 86.5% (macro-F1 0.775) for $\eta = 10^{-2}$, which is also where the train–validation gap is smallest. For $\eta \geq 10^{-1}$ the training accuracy saturates at 100% while validation drops to 83% and below: the network simply memorizes the 1280 training points. Conversely, $\eta = 10^{-3}$ leaves both curves low (val. 79.4%, F1 0.47) because, in 100 epochs, SGD has not yet converged. The learning rate is thus the single most influential knob, governing the *optimization* rather than the capacity.

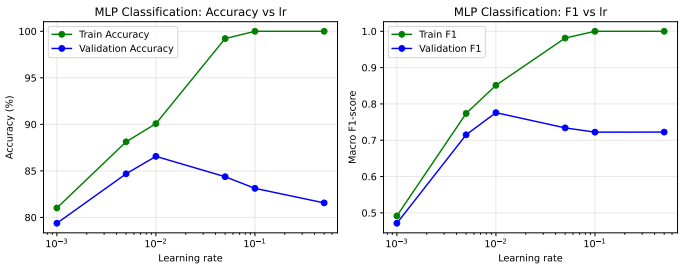


Figure 1. MLP classification: train (green) and validation (blue) accuracy (left) and macro-F1 (right) vs. learning rate. Best: $\eta = 10^{-2}$ (val. 86.5% / 0.775). High η drives the train curves to 100% while validation degrades – overfitting.

3.2. MLP: depth (regression)

Holding the width fixed and increasing the number of ReLU hidden layers from 1 to 14 ($\eta = 3 \cdot 10^{-3}$) makes the over-fitting explicit (Fig. 2): the training MSE drifts downward (from ≈ 0.94 toward 0.71) while the test MSE trends upward (from ≈ 1.05 to 1.22), so the generalization gap widens with depth. Small, shallow networks therefore generalize best: our architecture sweep selects $[8, 8]$ (validation MSE 0.965), which retrained attains test MSE 1.03 – a textbook bias–variance picture in which extra depth buys only variance.

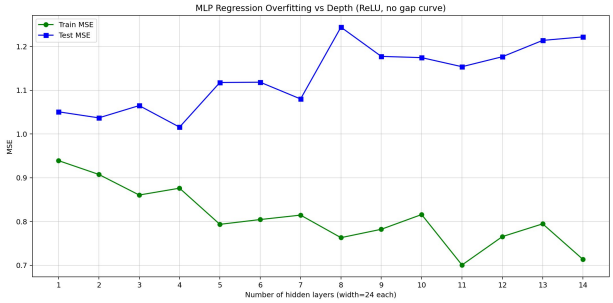


Figure 2. MLP regression over-fitting vs. depth (ReLU hidden units, $\eta = 3 \cdot 10^{-3}$). Train MSE (green) keeps falling with depth while test MSE (blue) trends upward: the gap grows. The shallowest network generalizes best.

3.3. K-Means: number of clusters

To choose a *structural* number of clusters we overlay the inertia J and the mean silhouette, both min–max-normalized, and take the K of largest vertical gap between them (Fig. 3). The gap peaks at $K^\star = 35$, where the inertia has already fallen to 0.44 while the silhouette is still 0.77 – the best compactness/separation trade-off. Yet the absolute silhouette never exceeds 0.06 (barely separated clusters), and $K^\star = 35$ still lies far below the F1-optimal $K = 149$: geometry and label purity disagree.

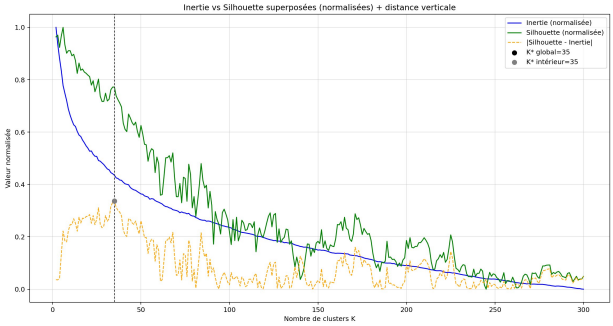


Figure 3. Structural-K selection: inertia and mean silhouette, each min–max normalized, with their absolute vertical gap (dashed). The gap peaks at $K^\star = 35$ (inertia 0.44, silhouette 0.77), far below the F1-optimal $K = 149$.

Fig. 4 reports accuracy and macro-F1 against $K \in \{2, \dots, 300\}$, each point averaged over 20 random seeds with its ± 1 std band. Averaging removes most of the single-seed noise: accuracy sits on a broad 71–73% plateau, while the macro-F1 – the metric that matters on this imbalanced problem – climbs from ≈ 0.31 at small K to a peak of 0.578 ± 0.04 at $K = 149$. More clusters give the majority-vote rule a finer label resolution, so the minority classes are no longer systematically outvoted as they are at small K (F1 ≈ 0.33 at $K = 10$).

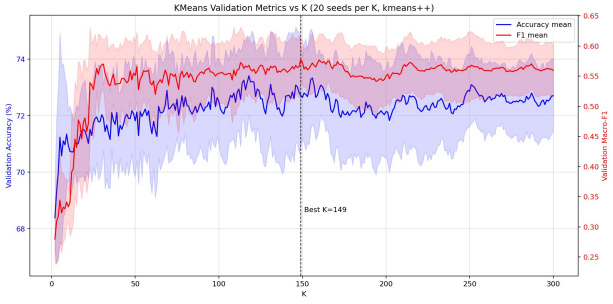


Figure 4. K-Means classification, mean $\pm$ std over 20 seeds (k-means++): validation accuracy (blue) and macro-F1 (red) vs. K. Macro-F1 peaks at $K = 149$ (0.578); accuracy holds a 71–73% plateau.

3.4. Final performance on the test set

We retrain the validation-selected configurations on the full 1600-sample set and evaluate them once on the test set (Table 1). The classifier – $[64, 32]$ at $\eta = 10^{-2}$, chosen on validation macro-F1 – reaches test accuracy 85.2% / F1 0.755. The regressor, a small two-layer net $[8, 8]$ ($\eta = 3 \cdot 10^{-3}$, val MSE 0.965), reaches test MSE 1.03, comfortably below Milestone 1's KNN regression (1.86). For K-Means the two ways of choosing K turn out to be practically equivalent: the unsupervised elbow gives $K^\star = 35$ (Fig. 3) with macro-F1 0.534, against 0.578 for the F1-optimal $K = 149$ (Fig. 4) – a gap small enough that the cheap, label-free elbow is just as good a choice. Retrained at $K^\star = 35$, K-Means reaches test accuracy 73.8% / F1 0.635. All configurations clear the project benchmarks.

Table 1. Test performance of the validation-selected models; Milestone 1's k-NN is included as a baseline.

Config	Task	Training	Test
MLP $[64, 32]$, $\eta = 0.01$	Classification	85.9% / 0.776	85.2% / 0.755
K-Means, $K^\star = 35$	Classification	71.2% / 0.534	73.8% / 0.635
MLP $[8, 8]$, $\eta = 0.003$	Regression	MSE 0.965	MSE 1.03
KNN, $k = 12$ (MS1)	Regression	MSE 2.15	MSE 1.86

Note: ***Test*** *= held-out test set. Milestone 1's logistic regression reached 88.75% / 0.863 for reference.*

4. DISCUSSION AND CONCLUSION

Bias–variance, revisited. The two MLP studies reproduce the central lesson of Milestone 1 from two angles. For the learning rate (Fig. 1) the failure modes are *optimization*-driven: too small and the network under-fits because SGD has not converged in 100 epochs; too large and it over-fits by memorizing the training set (train accuracy 100%, validation collapsing). For depth (Fig. 2) the failure is *capacity*-driven and identical in shape to the KNN curves of Milestone 1: the training MSE keeps dropping while the test MSE trends upward as soon as the extra layers can fit the noise. The best models are deliberately modest – $\eta = 10^{-2}$ for classification and a small, shallow network for regression.

Clustering as a classifier. K-Means exposes a different difficulty: its objective (geometric compactness, Equation 2) is not the quantity we ultimately care about (label purity). Hence the structural K flagged by the inertia–silhouette gap ($K^\star \approx 35$, Fig. 3) sits far below the $K = 149$ that maximizes macro-F1 (Fig. 4). Raising K is the only lever the majority-vote rule has to recover the minority classes, which is why F1 keeps climbing long after the geometric gain has saturated. Even averaged over 20 seeds the metric bands stay wide, confirming that K-Means is far more sensitive to initialization than the MLP.

Why depth does not help here. An activation ablation is telling: replacing the ReLU hidden units by the identity – an essentially linear model – still reaches 84.4% accuracy, within two points of ReLU (86.5%) and above sigmoid (82.2%). With this and the depth-driven over-fitting on regression, the feature-to-target map is essentially linear: extra capacity buys variance, not signal. The fix is regularization and feature engineering, not larger models.